COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/2/26	**DATE SUBMITTED:**	09/2/26
NAME:	Jiro Luis A. Katindig	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

Table 7-1. Array of Values for Sort Algorithm Testing

ACTIVITY 7 > main.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);

    // std::cout << "\nBubble Sort: \n" << std::endl;
    // bubbleSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    // std::cout << "\nSelection Sort: \n" << std::endl;
    // selectionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    // std::cout << "\nInsertion Sort: \n" << std::endl;
    // insertionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

```
Unsorted Array:

51 63 50 5 31 46 79 80 90 65 81 65 35 91 81 85 50 10 23 1 31 17 42 52 42 23 76 89 92 92 70 13 49 3 11 12 23 37 1 23 37 1 41 98 17 16 1 31 31 87 83 59 28 27 11 75 14 52 15 74 56 73 3 69 60 71 8 55 43 11 21 34
68 89 35 25 81 59 60 39 6 79 59 40 4 60 2 55 49 22 6 28 85 47 50 55 44 20 37 32
```

Observations

The program generated an array of 100 random values ranging from 0 to 99 using rand() with time as the seed. Since the array was not yet processed by any sorting algorithm, the values appeared in a completely random order with no pattern. This unsorted array served as the base data that would later be used to test and compare the bubble, selection, and insertion sort algorithms.

Table 7-2. Bubble Sort Technique

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);

    std::cout << "\nBubble Sort: \n" << std::endl;
    bubbleSort(unsortedArr, maxSize);
    displayArray(unsortedArr);

    // std::cout << "\nSelection Sort: \n" << std::endl;
    // selectionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    // std::cout << "\nInsertion Sort: \n" << std::endl;
    // insertionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

```cpp
template<typename T>
void bubbleSort(T arr[], size_t arrSize){
    //step 1 for i = 0 to N-1 repeat inner loop
    for(int i = 0; i < arrSize; i++){
        //step 2 inner loop that for K = i + 1 to N -I repeat
        for(int j = 0; j < arrSize - i - 1; j++){
            //step 3 swap if the arr[j] > arr[i]
            if (arr[j] > arr[j+1]){
                //swap
                T temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

```
Unsorted Array:

39 50 34 57 9 18 31 24 18 24 34 41 76 8 12 41 38 98 86 88 87 52 26 44 52 56 52 57 37 6 9 70 95 92 62 68 89 67 39 70 92 50 81 80 53 69 24 12 69 3 57 25 19 26 73 86 25 55 6 75 18 33 38 30 76 7 81 1 50 34 70 89
76 87 84 94 37 37 27 70 93 69 38 28 43 19 90 93 49 4 41 24 87 96 60 69 88 22 69 11

Bubble Sort:

1 3 4 6 6 7 8 9 9 11 12 12 18 18 18 19 19 22 24 24 24 24 25 25 26 26 27 28 30 31 33 34 34 34 37 37 37 38 38 38 39 39 41 41 41 43 44 49 50 50 50 52 52 52 53 55 56 57 57 57 60 62 67 68 69 69 69 69 70 70 70 7
0 73 75 76 76 76 80 81 81 84 86 86 86 87 87 87 88 88 89 89 90 92 92 93 93 94 95 96 98
```

Observations

Using the same unsorted array, bubble sort was applied and it successfully arranged the values from smallest to largest. The algorithm worked by comparing each pair of adjacent elements and swapping them whenever the left value was greater than the right value, repeating this process across multiple passes until no more swaps were needed. This showed that bubble sort is easy to understand and implement, but it also required a lot of comparisons and swaps since it checks the whole array repeatedly, making it less efficient for larger datasets.

Table 7-3. Selection Sort Algorithm

ACTIVITY 7 > main.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);

    // std::cout << "\nBubble Sort: \n" << std::endl;
    // bubbleSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    std::cout << "\nSelection Sort: \n" << std::endl;
    selectionSort(unsortedArr, maxSize);
    displayArray(unsortedArr);

    // std::cout << "\nInsertion Sort: \n" << std::endl;
    // insertionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

```cpp
template<typename T>
//array, current position, size of array
int routineSmallest(T arr[], int K, const int arrSize){
    int position;
    T smallestElem = arr[K];

    for(int j = K+1; j < arrSize; j++){
        if(arr[j] < smallestElem){
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}


template<typename T>
void selectionSort(T arr[], size_t arrSize){
    int POS, temp, pass;
    for(int i=0; i < arrSize; i++){
        POS = routineSmallest(arr, i,arrSize);
        //swapping:
        temp = arr[i];

        arr[i] = arr[POS];
        arr[POS] = temp;

        //for verification and coutning of number of passes that was done
        pass++;

    }
}
```

```
Unsorted Array:

71 43 16 88 83 81 26 9 69 46 63 92 77 35 83 22 11 65 70 48 7 96 52 1 3 23 13 62 85 90 36 10 90 65 39 38 45 53 48 96 68 97 82 36 96 79 87 67 43 14 29 44 15 33 18 7 24 73 16 92 18 76 41 11 76 43 0 89 52 58 28 9
5 44 37 8 22 88 97 55 19 94 54 97 51 6 43 99 86 22 37 35 7 77 5 82 32 61 37 10 4

Selection Sort:

0 1 3 4 5 6 7 7 7 8 9 10 10 11 11 13 14 15 16 16 18 18 19 22 22 22 23 24 26 28 29 32 33 35 35 36 36 37 37 37 38 39 41 43 43 43 43 44 44 45 46 48 48 51 52 52 53 54 55 58 61 62 63 65 65 67 68 69 70 71 73 76 76
77 77 79 81 82 82 83 83 85 86 87 90 88 88 89 97 90 92 92 94 95 97 96 96 96 99 97
```

Table 7-4. Insertion Sort Algorithm

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);

    // std::cout << "\nBubble Sort: \n" << std::endl;
    // bubbleSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    // std::cout << "\nSelection Sort: \n" << std::endl;
    // selectionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    std::cout << "\nInsertion Sort: \n" << std::endl;
    insertionSort(unsortedArr, maxSize);
    displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

```cpp
template<typename T>
void insertionSort( T arr[], size_t arrSize){
    int K = 0, J, temp;

    while(K < arrSize){
        temp = arr[K];
        J = K - 1;

        while(temp <= arr[J]){
            arr[J + 1] = arr[J];
            J--;
        }

        arr[J + 1] = temp;
        K++;
    }
}
```

```
Unsorted Array:

20 88 42 59 99 68 11 6 7 65 98 57 33 86 43 62 8 68 35 1 1 11 14 82 43 64 74 23 37 86 32 18 86 59 81 25 55 70 36 35 72 98 53 98 72 0 9 15 78 37 49 98 60 34 8 59 93 52 93 23 44 80 72 47 90 75 68 46 76 53 37 69
13 31 15 80 17 7 70 7 19 31 7 42 76 72 43 28 53 28 19 78 21 21 0 55 38 77 51 75

Insertion Sort:

0 1 1 6 7 7 7 7 8 8 9 11 11 13 14 15 15 17 18 19 19 20 21 21 23 23 25 28 28 31 31 32 33 34 35 35 36 37 37 37 38 42 42 43 43 43 44 46 47 49 51 52 53 53 53 55 55 57 59 59 59 60 62 64 65 68 68 68 69 70 70 72 72
72 72 74 75 75 76 76 77 78 78 80 80 81 82 86 86 86 88 90 93 93 98 98 98 98 99 596
```

Observations

Insertion sort produced the same correctly sorted array as the other two methods. It worked by taking each value one at a time and shifting the elements greater than it to the right until it found the correct spot to insert it, gradually building a sorted section of the array from left to right. This made it efficient for data that is already partially sorted, since fewer shifts are needed in that case.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ACTIVITY 7 > suppmain.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "suppAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    std::string candidates[5] = {
        "John Michael Dela Cruz",
        "Jane Marie Santos",
        "Mark Anthony Reyes",
        "Luis Miguel Ramos",
        "Anna Patricia Villanueva"
    };
    int votes[maxSize];
    srand(time(0));

    //generate random votes
    for (int i = 0; i < maxSize; i++) {
        votes[i] = (rand() % 5) + 1;
    }

    std::cout << "Unsorted votes: \n";
    displayArray(votes);

    //sort votes
    selectionSort(votes, maxSize);
    std::cout << "\nSorted votes: \n";
    displayArray(votes);

    //count and tally
    int voteCounts[5] = {0};
    int winnerIndex = 0;

    for (int i = 0; i < maxSize; i++) {
        voteCounts[votes[i] - 1]++;
    }

    std::cout << "\nVote total: \n";
    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << (i + 1) << " (" << candidates[i]
                  << "): " << voteCounts[i] << " votes\n";

        if (voteCounts[i] > voteCounts[winnerIndex]) {
            winnerIndex = i;
        }
    }
    std::cout << "\nWinning Candidate: " << candidates[winnerIndex]
              << " with " << voteCounts[winnerIndex] << " votes\n";

    return 0;
}
```

ACTIVITY 7 > C suppAlgo.h > ...

```
#ifndef SUPPALGO_H
#define SUPPALGO_H
#include <cstddef>

template <typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position = K;
    T smallestElem = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize) {
    int pos;
    T temp;

    for (size_t i = 0; i < arrSize; i++) {
        pos = routineSmallest(arr, i, arrSize);
        temp = arr[i];
        arr[i] = arr[pos];
        arr[pos] = temp;
    }
}

#endif
```

```
Unsorted votes:
1 2 4 2 1 3 5 4 2 3 1 1 1 5 2 4 4 3 3 5 5 2 4 3 2 4 4 3 4 2 5 5 1 1 4 3 4 4 4 3 4 1 5 1 5 1 1 1 2 4 4 1 4 2 5 3 1 1 5 3 2 2 2 4 1 3 4 2 3 1 4 4 3 5 5 2 3 5 4 4 3 1 4 5 1 1 1 4 2 4 4 3 2 5 1 5 2 1 3 3

Sorted votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote total:
Candidate 1 (John Michael Dela Cruz): 23 votes
Candidate 2 (Jane Marie Santos): 17 votes
Candidate 3 (Mark Anthony Reyes): 18 votes
Candidate 4 (Luis Miguel Ramos): 26 votes
Candidate 5 (Anna Patricia Villanueva): 16 votes

Winning Candidate: Luis Miguel Ramos with 26 votes
```

Observations

For the supplementary activity, selection sort was used to sort 100 randomly generated votes ranging from 1 to 5. After sorting, identical vote values were grouped together, which made it easy for the program to count how many votes each candidate received. The program correctly tallied the votes for all five candidates and identified the candidate with the highest number of votes as the winner, based on the actual run shown in the output.

Conclusion

Through this activity, I was able to understand how bubble sort, selection sort, and insertion sort work by implementing all three in C++ and observing how each one rearranges an unsorted array. Even though all three algorithms produced the same correctly sorted output, I noticed they go about it differently, with bubble sort relying on repeated swaps, selection sort minimizing swaps by picking the smallest value each pass, and insertion sort building the sorted array gradually through shifting. Applying selection sort to the vote counting problem also helped me see how sorting can support a practical task like tallying and comparing results. Overall, this activity helped me better understand not just how these algorithms function, but also how the choice of algorithm can affect efficiency depending on the situation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.